รายงาน Stored Procedure ในฐานข้อมูลระบบ UP Cloth Market

รายวิชา 227202 Database · กลุ่ม “ไม่ใส่กุ้ง”

ฐานข้อมูล: Supabase (PostgreSQL)

1. ภาพรวม

Stored Procedure คือชุดคำสั่งที่ตั้งชื่อและเก็บไว้ในฐานข้อมูลเหมือน function แต่ใน PostgreSQL เป็นวัตถุคนละชนิด สร้างด้วย CREATE PROCEDURE (มีตั้งแต่ PostgreSQL 11) เรียกด้วยคำสั่ง CALL ไม่มีค่าส่งกลับแบบ function แต่ส่งผลกลับได้ทางพารามิเตอร์ชนิด INOUT ใช้กับงานที่ “สั่งให้ทำ” มากกว่างานที่ “ถามค่า” เช่น งานบำรุงรักษาข้อมูลเป็นชุด

ก่อนเริ่มทำรายงานนี้ กลุ่มตรวจฐานข้อมูลแล้วพบว่ายังไม่มี procedure เลยแม้แต่ตัวเดียว งานทั้งหมดเขียนเป็น function 4 ตัว (notify_user, refresh_product_quantity, place_order, is_own_order) เพราะงานที่หน้าเว็บสั่งต้องเรียกผ่าน REST API ของ Supabase (PostgREST) ซึ่งเปิดให้เรียกเฉพาะ function ที่ /rest/v1/rpc/... ส่วน procedure เรียกได้จาก SQL โดยตรงเท่านั้น

รายงานนี้จึงบันทึกผลการตรวจ (ว่าไม่มีเพราะอะไร) และออกแบบ candidate สำหรับงานที่เหมาะจะเป็น procedure ในอนาคต โดยไม่มีการสร้างอะไรลงฐานข้อมูลจริง

#	Procedure	พารามิเตอร์	ทำอะไร	ตารางที่แก้	หมายเหตุ
—	(ยังไม่มี)	—	—	—	ตรวจแล้วว่าไม่มี procedure ใน schema public (หัวข้อ 4)

[FIG 1: ผล query pg_proc ว่าไม่มี procedure (prokind = 'p') ได้ 0 แถว คำสั่งดู: select proname from pg_proc where pronamespace = 'public'::regnamespace and prokind = 'p' order by 1;]

1.1 Procedure ต่างจาก Function อย่างไร

หัวข้อ	FUNCTION	PROCEDURE
คำสั่งสร้าง	CREATE FUNCTION ... RETURNS ...	CREATE PROCEDURE ... ไม่มี RETURNS
วิธีเรียก	SELECT f() หรือใช้ในนิพจน์ เช่น WHERE และนโยบาย RLS	CALL p() เท่านั้น ถ้าใช้ SELECT จะ error
ค่าที่ส่งกลับ	ค่าเดียว แถว หรือหลายแถว (SETOF)	ไม่มี return ส่งผลกลับผ่านพารามิเตอร์ INOUT/OUT
ใช้ในคำสั่ง SQL อื่น	ได้	ไม่ได้
ควบคุมทรานแซกชัน	COMMIT/ROLLBACK ข้างในไม่ได้	ทำได้ เมื่อ CALL จากนอกทรานแซกชัน
เรียกผ่าน Supabase REST API	ได้ที่ /rest/v1/rpc/ชื่อฟังก์ชัน	ไม่ได้
ในระบบนี้	4 ตัว (รายงาน Trigger และรายงาน SQL Function)	0 ตัว (รายงานนี้)

2. ทำไมระบบนี้จึงไม่มี procedure (วิเคราะห์ทีละ routine)

Routine ปัจจุบัน	ชนิด	เหตุผลที่ต้องเป็น function เปลี่ยนเป็น procedure ไม่ได้
place_order(uuid, jsonb, text)	RPC ที่หน้าเว็บเรียก	ต้องเปิดเป็น REST API (/rpc/place_order) และต้องคืน order_id ให้หน้าเว็บพาไปหน้ารายละเอียด ถ้าเป็น procedure หน้าเว็บจะเรียกไม่ได้เลย
is_own_order(int)	helper ที่ RLS เรียก	ถูกใช้ในนิพจน์ USING/WITH CHECK ของ policy (using (is_own_order(order_id))) procedure ใช้ในนิพจน์ไม่ได้
notify_user()	trigger function	ถูกเรียกจาก trigger 4 ตัว (trg_order_item_notify, trg_orders_status_notify, trg_orders_insert_notify, trg_messages_notify) ตามนิยาม PostgreSQL ต้องเป็น function ที่ RETURNS trigger เท่านั้น
refresh_product_quantity()	trigger function	เหตุผลเดียวกับข้างบน ถูกเรียกจาก trg_refresh_product_quantity

สรุป: ทุก routine ในระบบนี้ผูกกับกลไกที่บังคับว่าต้องเป็น function (REST / RLS / trigger) จึงไม่มีงานเหลือให้เป็น procedure โดยธรรมชาติ

3. Candidate ที่เหมาะจะเป็น procedure ในอนาคต (ออกแบบบนกระดาษ ยังไม่สร้างจริง)

เกณฑ์คัดเลือก: งานที่ผู้ดูแลระบบหรือตัวตั้งเวลาสั่ง ไม่ใช่ผู้ใช้กดจากหน้าเว็บ, ไม่ต้องคืนค่าให้หน้าเว็บ, ทำเป็นชุดแล้วจบในครั้งเดียว

#	Procedure (ชื่อเสนอ)	พารามิเตอร์	ทำอะไร	ตารางที่แก้
1	purge_read_notifications	IN p_older_than_days (default 30), INOUT p_deleted	ลบแจ้งเตือนที่อ่านแล้วและเก่าเกิน N วัน (ตารางโตไม่หยุดเพราะทุกออเดอร์/แชทสร้างแถว)	notifications (DELETE)
2	reconcile_product_quantities	INOUT p_fixed	ตรวจทุกสินค้าว่า quantity ตรงกับผลรวม variants ไหม ถ้าไม่ตรงให้แก้ (ตาข่ายนิรภัยของ trigger ข้อ 4)	product (UPDATE เฉพาะแถวที่เพี้ยน)

เหตุผลที่ 2 ตัวนี้เหมาะเป็น procedure ไม่ใช่ function: - ไม่มีใครต้องเรียกผ่าน REST (งานหลังบ้านล้วน) และไม่ต้องคืนค่าเป็นชุดข้อมูล (แค่จำนวนแถวผ่าน INOUT) - ควรตัดสิทธิ์ EXECUTE ของ anon/authenticated ให้เหลือผู้ดูแลระบบ (รันผ่าน SQL Editor) ซึ่งทำได้กับ procedure โดยไม่กระทบหน้าเว็บเลย - ตั้งเวลาได้ด้วย pg_cron ในอนาคต (SELECT cron.schedule(..., 'CALL ...'))

ข้อควรระวังถ้าจะสร้างจริง: - purge_read_notifications ต้องลบเฉพาะ is_read = true เท่านั้น ห้ามแตะแถวที่ยังไม่อ่าน - ทั้งคู่ต้องมี SET search_path TO 'public' และห้ามเป็น SECURITY DEFINER

(รันด้วยสิทธิ์ผู้เรียกซึ่งเป็นผู้ดูแลระบบอยู่แล้ว)

4. สรุป

หัวข้อ	ผล
จำนวน procedure ในระบบ	0 ตัว (ตรวจจาก repo + query pg_proc)
จำนวน function ในระบบ	4 ตัว (notify_user, refresh_product_quantity, place_order, is_own_order)
เหตุผล	ทุกงานผูกกับกลไกที่ต้องเป็น function (REST / RLS / trigger)
งานที่เหมาะเป็น procedure ในอนาคต	2 ตัว (หัวข้อ 3) สถานะ: ออกแบบแล้ว ยังไม่สร้าง

5. วิธีการตรวจสอบ

- ตรวจว่าไม่มี procedure: select proname from pg_proc where pronamespace = 'public'::regnamespace and prokind = 'p' order by 1; (ต้องได้ 0 แถว) บนฐานข้อมูลจริง ส่วน prokind = 'f' ต้องได้ 4 ชื่อตรงตารางข้างบน
- ตรวจใน repo: grep -r "CREATE.*PROCEDURE" supabase/ ต้องไม่เจออะไรเลย (ตรวจแล้วเมื่อเขียนรายงานนี้)
- ภาพหน้าจอแอปไม่เกี่ยวข้องกับรายงานนี้ (ไม่มีฟีเจอร์ที่เรียก procedure)

ภาคผนวก: รายชื่อไฟล์ภาพ

ภาพที่	ไฟล์	เนื้อหา
1	fig01_no_procedures.png	ผล query pg_proc ว่าไม่มี procedure (0 แถว) ผ่าน SQL Editor